so that the results return the `synopsis` and `rating` fields along with all fields with `title` as a prefix.

Listing 8.30 Using the `_source` tag to fetch custom fields

```
GET movies/_search
{
  "_source": ["title*","synopsis", "rating"],
  "query": {
    "match": {
      "certificate": "R"
    }
  }
}
```

We can take the `_source` option even further by setting a list of `includes` and `excludes` to control the return fields.

Listing 8.31 Source filtering using `includes` and `excludes`

```
GET movies/_search
{
  "_source": {
    "includes": ["title*","synopsis","genre"],
    "excludes": ["title.original"]
  },
  "query": {
    "match": {
      "certificate": "R"
    }
  }
}
```

The `_source` object expects two arrays:

- `includes`—All the fields that should be returned in the result
- `excludes`—Fields that must be excluded from the fields returned from the `includes` list

In listing 8.31, we expect all `title` fields (`title` and `title.original`), as well as the `synopsis` and `genre` fields, to be returned from the query. However, we can suppress `title.original` by including it in the `excludes` array. We can play with the `includes` and `excludes` arrays to gain finer control over what fields are returned and suppressed. For example, if we add an `"excludes": ["synopsis","actors"]` array to the `_source` object, all fields except `synopsis` and `actors` are returned.

8.8.6 Searching across indexes and data streams

Data is usually spread across indexes and data streams. Fortunately, Elasticsearch lets us search data across multiple indexes and data streams by appending the required